Virtual Private Networks (VPN)

What a VPN is, the kinds of VPN, and their working

Roshan and Devashish
IIT Bombay

Learning

01

VPN basics

What a VPN is, what it protects and how it is built

02

Uses of VPN

Remote access and site-to-site

03

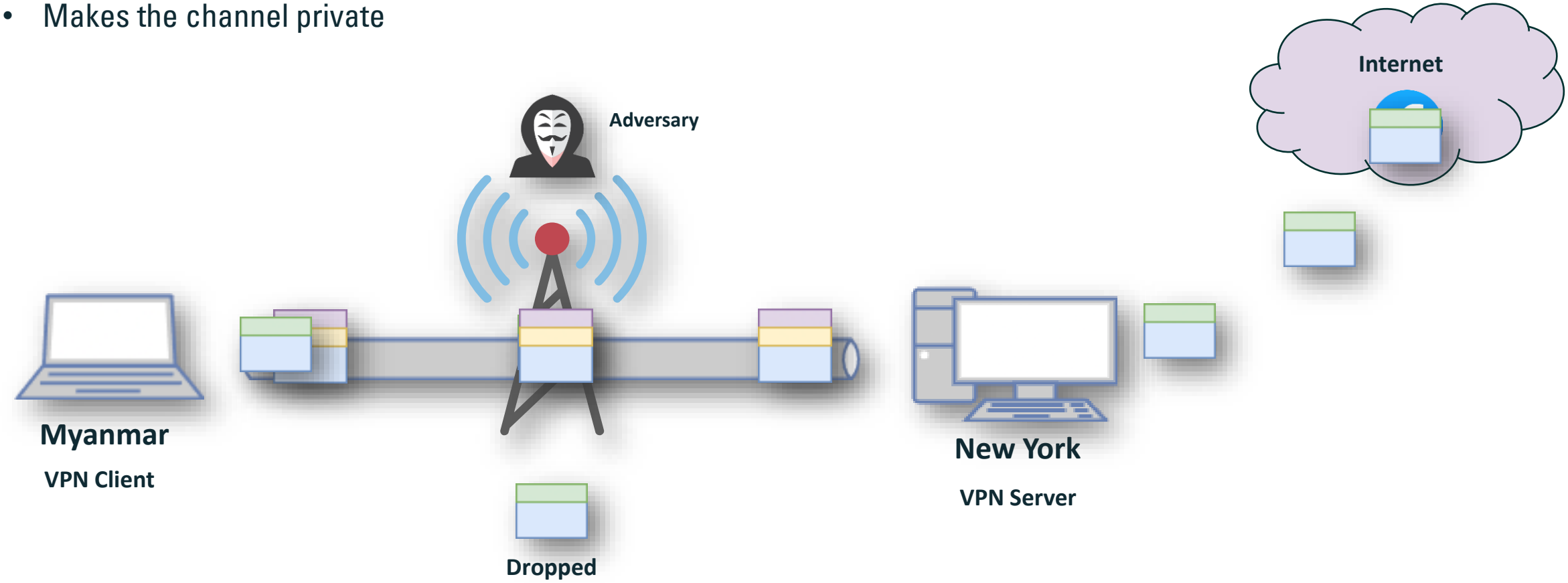
Protocols

How OpenVPN and WireGuard work, step by step, down to the packets

Introduction

What is a VPN?

- A privacy enhancing tool
- Makes the channel private



Confidentiality

Nobody in the middle can read your traffic.

Integrity

If someone changes the data on the way, it is noticed and dropped.

Authentication

Both ends prove who they are before the tunnel starts.

Use-case of the VPN

Remote access VPN (client-to-site)



Site-to-site VPN (network-to-network)



Always on, from gateway to gateway. Users need no VPN software.

VPN protocols

How a tunnel is set up

VPN protocols at a glance

IPsec



1995 onward

Built into most operating systems and routers.

Motivation

- IP itself had no security.
- IPsec adds encryption and authentication at the IP layer, so every application is protected with no changes, and vendors can interoperate.

OpenVPN

2001, James Yonan

Open source, uses TLS. Flexible, and gets through most firewalls.

Motivation

- IPsec was kernel level, complex to set up and often blocked by NAT and firewalls.
- OpenVPN reused the TLS library everyone already trusted and ran as an ordinary program.

WireGuard

2016, Jason Donenfeld

Small and fast, with a fixed set of modern ciphers.

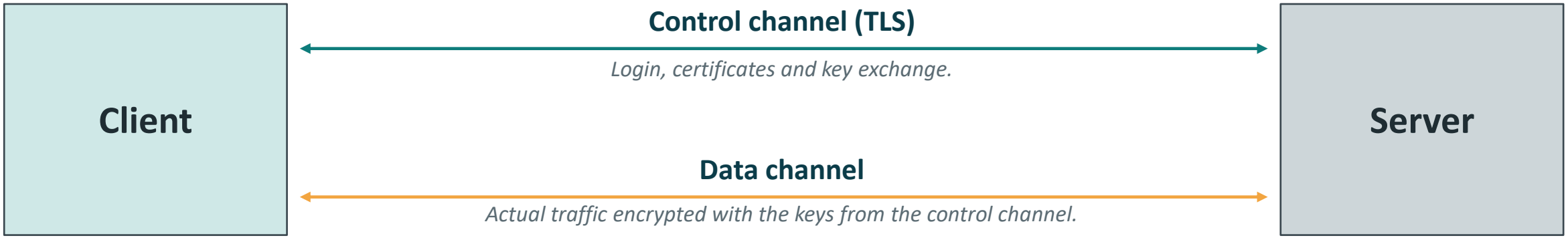
Motivation

- IPsec and OpenVPN are big, with many options and cipher negotiation. That means a large attack surface and a lot to audit and misconfigure.



OpenVPN

OpenVPN overview



UDP or TCP and Port: 1194

Open source

Since 2001. A normal program runs in User space.

TLS 1.2 / 1.3

Known protocol for handshake and key derivation.

tls-crypt / tls-auth

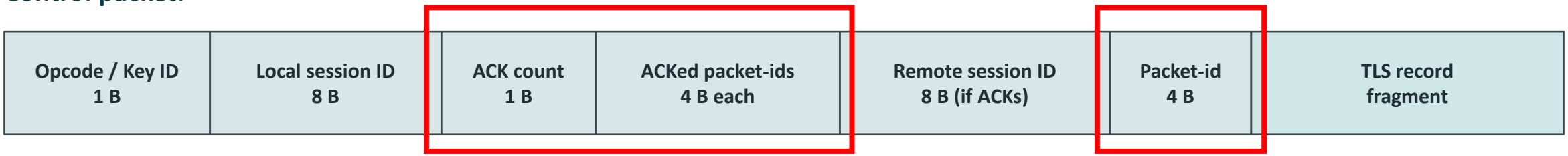
Protects the handshake from scanners and floods.

Can work where UDP fails

Uses TCP 443 to get through strict firewalls.

OpenVPN packet formats

Control packet:



Opcode / Key ID: Upper 5 bits are the packet type (e.g. P_CONTROL_V1, P_ACK_V1, the hard-reset types). Lower 3 bits are the key ID.
Local session ID: Random ID the sender picks for this session.
ACK count: How many packet-ids are acknowledged in this packet.
ACKed packet-ids: The control packets the sender has received from the server. First packet will have 0.
Remote session ID: The server's session ID. First packet will have 0.
Packet-id: Sequence number of this control packet, used for ordering and retransmission.
TLS record: A fragment of the TLS handshake or records.

Q. Why do we need the ACK headers in the control packet?

Because TLS messages require in-order delivery.

OpenVPN packet formats

Data packet:

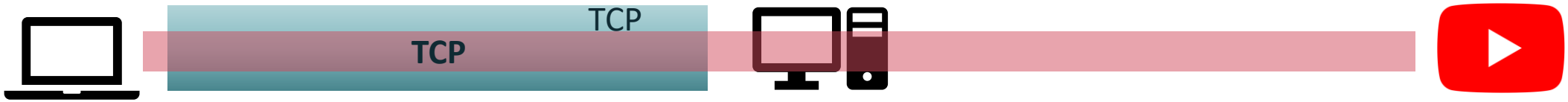
Opcode / Key ID 1 B	Peer ID 3 B	Packet-id 4 B	Auth tag 16 B	Encrypted inner IP packet

Opcode / Key ID: Packet type and key ID.
Peer ID: Lets the server identify the client without relying on its source IP and port, so the client can roam.
Packet-id: Counter used for replay protection.
Auth tag: The GCM authentication tag.
Encrypted payload: The inner IP packet.

Why there is no ACK count and ACK packet id field?

The inner traffic is mostly TCP.

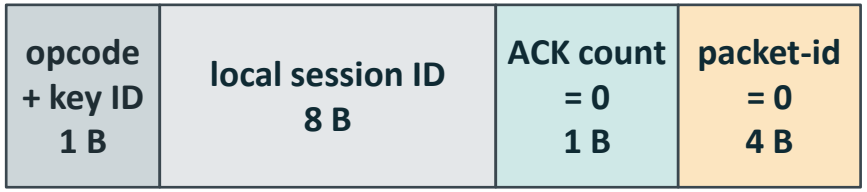
TCP inside TCP tunnel type problem: TCP traffic inside the TCP tunnel will have multiple retransmissions.



Opcodes 7 and 8: starting a session

Opcode 7: client to server

P_CONTROL_HARD_RESET_CLIENT_V2



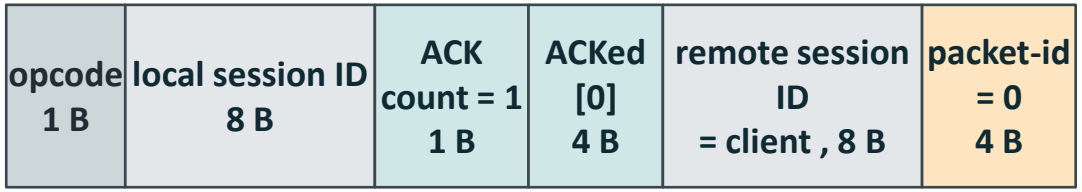
Sent by: the client, as the very first packet of a connection.

Means: "New session. Throw away any old state and start from scratch."

Nothing to ACK yet, so there is no remote session ID.

Opcode 8: server to client

P_CONTROL_HARD_RESET_SERVER_V2



Sent by: the server, in reply to a valid opcode 7.

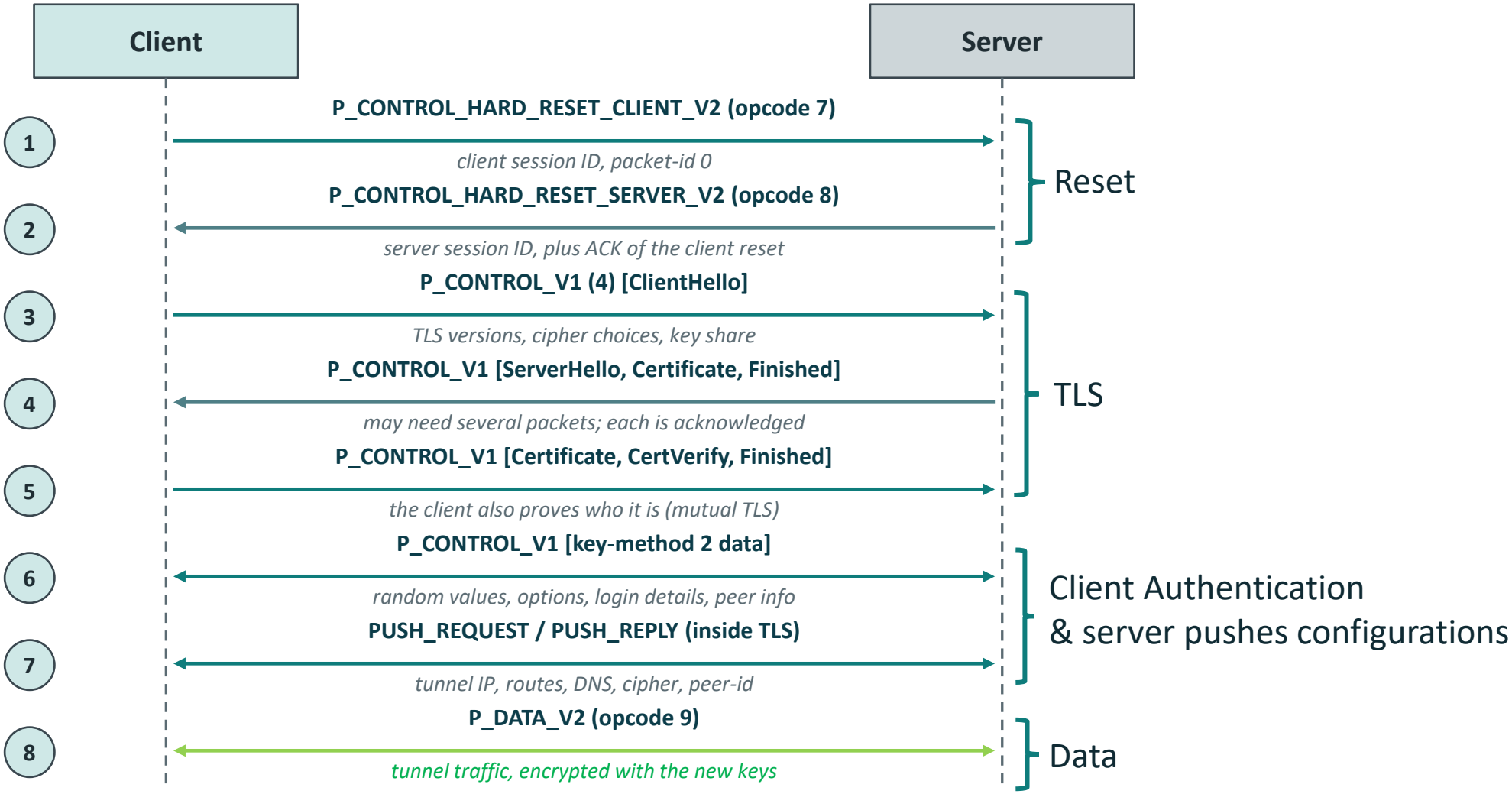
Means: "Got your packet 0, and here is my own session ID."

It starts the server's own numbering at packet-id 0.

Why "hard" reset?

- UDP has no connection, so the server cannot tell a brand new client from a stale or repeated packet.
- A hard reset says: forget the old control channel state, this is a new session. The session IDs and the packet numbers (starting at 0) are agreed in these two packets.

OpenVPN handshake, message by message



Before the VPN: how the PC routes packets

Routing table of the client (no VPN)

```
$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Iface
0.0.0.0         192.168.1.1     0.0.0.0         UG    600    wlan0
192.168.1.0     0.0.0.0         255.255.255.0   U     600    wlan0

# Destination 0.0.0.0 with Genmask 0.0.0.0 is the default route.
# Flags: U = up, G = via a gateway (router), H = one host.
# Gateway 0.0.0.0 means the network is directly connected.
```

Longest match wins. The default route is only used when nothing else matches.

Packet to	Route used	Goes out via
142.250.195.78 (YouTube)	default	wlan0, via home router
192.168.1.20 (printer)	192.168.1.0/24	wlan0, direct

How the kernel matches a route

Destination	Gateway	Genmask	Flags	Metric	Iface
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0

A route matches when `(destination IP AND Genmask) = route Destination.` If several match, the longest prefix wins.

Packet to 142.250.195.78 (YouTube)

Route 1: default 0.0.0.0/0 (0.0.0.0)

```
dest  10001110.11111010.11000011.01001110
mask  00000000.00000000.00000000.00000000
AND   00000000.00000000.00000000.00000000  = 0.0.0.0
```

0.0.0.0 equals 0.0.0.0: match, prefix /0

Route 2: 192.168.1.0/24 (255.255.255.0)

```
dest  10001110.11111010.11000011.01001110
mask  11111111.11111111.11111111.00000000
AND   10001110.11111010.11000011.00000000  = 142.250.195.0
```

142.250.195.0 is not 192.168.1.0: no match

Only the default route matches
Gateway 192.168.1.1 via the home router

Longest prefix match: the most specific route wins

1	**Find every route that matches** `(destination AND Genmask) = route Destination`
2	**Pick the longest prefix** The route that fixes the most leading bits is the most specific, so it wins.
3	**Metric only breaks ties** It is compared only between routes with the same prefix length.

A longer prefix covers a narrower range of addresses

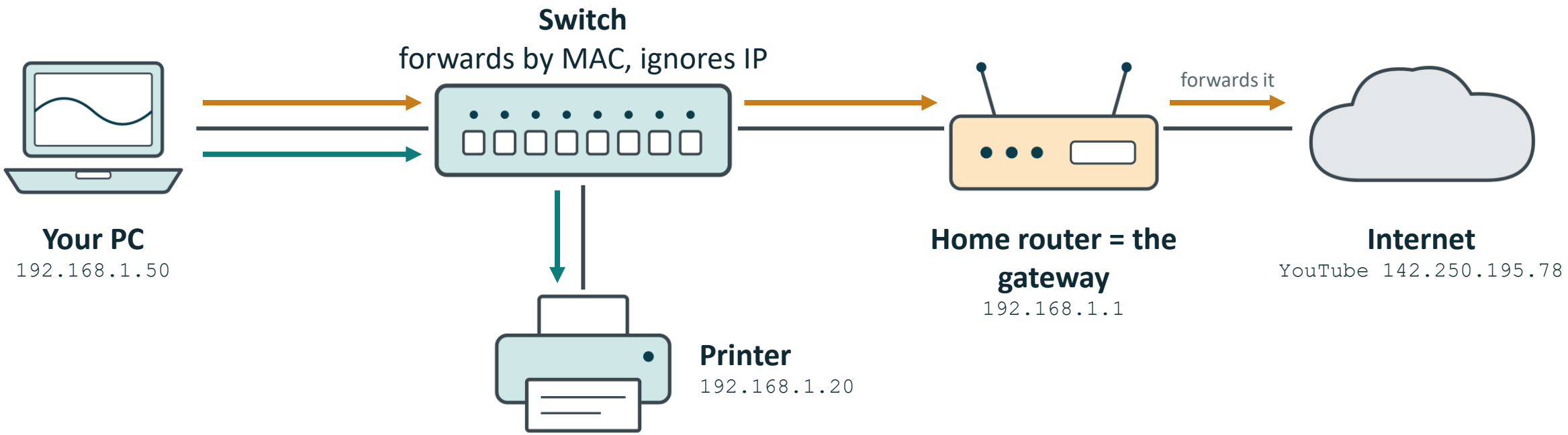
0.0.0.0/0	every address (default route)
192.168.1.0/24	256 addresses
192.168.1.20/32	1 address

Packet to	Routes that match	Route used
192.168.1.20	All 3 from above	**192.168.1.20/32 (the longest)**

The next slides use this rule: the default route is only the fallback, and a more specific route always overrides it.

Printer vs gateway: who the PC sends the frame to

Path A: packet for the printer Path B: packet for YouTube



Path A: the printer is on my own network

Route 192.168.1.0/24, no gateway. ARP finds the printer's MAC, and the frame goes to it. The switch delivers it by MAC. The router never sees it.

Path B: YouTube is not on my network

Default route, gateway 192.168.1.1. ARP finds the router's MAC, so dest MAC = router while dest IP stays 142.250.195.78. The switch passes it to the router, which sends it onwards.

Routing Table: longest prefix match

Destination	Gateway	Genmask	Flags	Metric	Iface
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0

A route matches when `(destination IP AND Genmask) = route Destination.` If several match, the longest prefix wins.

Packet to 192.168.1.20 (printer)

Route 1: default 0.0.0.0/0 (0.0.0.0)

```
dest  11000000.10101000.00000001.00010100
mask  00000000.00000000.00000000.00000000
AND   00000000.00000000.00000000.00000000  = 0.0.0.0
```

0.0.0.0 equals 0.0.0.0: match, prefix /0

Route 2: 192.168.1.0/24 (255.255.255.0)

```
dest  11000000.10101000.00000001.00010100
mask  11111111.11111111.11111111.00000000
AND   11000000.10101000.00000001.00000000  = 192.168.1.0
```

192.168.1.0 equals 192.168.1.0: match, prefix /24

Both match, so the longest prefix wins: /24 beats /0
Gateway 0.0.0.0 straight to the printer

Metric is only a tie-breaker between routes with the same prefix length. It never overrides a longer prefix.

After connecting to the VPN

VPN Server: 198.51.100.7

Routing table of the client

$ route -n
Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Iface	
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0	(1)
0.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(2)
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0	(3)
128.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(4)
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0	(5)
198.51.100.7	192.168.1.1	255.255.255.255	UGH	0	wlan0	(6)

```
# Blue rows were added by the VPN.
# Genmask 128.0.0.0 = /1, so the
two /1 routes cover every address.
```

0.0.0.0/1 = 0.0.0.0 to 127.255.255.255
128.0.0.0/1 = 128.0.0.0 to 255.255.255.255

Together they cover every IPv4 address and send it to VPN server.

After connecting to the VPN

VPN Server: 198.51.100.7
YouTube: 142.250.195.78
Client IP: 192.10.90.100

$ route -n
Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Iface	
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0	(1)
0.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(2)
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0	(3)
128.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(4)
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0	(5)
198.51.100.7	192.168.1.1	255.255.255.255	UGH	0	wlan0	(6)

0.0.0.0/1 = 0.0.0.0 to 127.255.255.255
128.0.0.0/1 = 128.0.0.0 to 255.255.255.255

Step 1: The browser sends the packet to YouTube (142.250.195.78).

Rows 1 and 4 match. Row 4 has the longer prefix (**/1** vs **/0**), so it wins: send via gateway **10.8.0.1** out of **tun0**.

The packet leaves with the source address of **tun0**:
Inner packet: 10.8.0.6 → 142.250.195.78

Matching the YouTube packet after the VPN is up

Packet to 142.250.195.78 (YouTube) `10001110.11111010.11000011.01001110`

The first bit is 1, so the address is above 127.255.255.255: it can only fit the 128.0.0.0/1 half, never 0.0.0.0/1.

(1) 0.0.0.0 / 0 (default)

dest	10001110.11111010.11000011.01001110
mask	00000000.00000000.00000000.00000000
AND	**00000000.00000000.00000000.00000000**

0.0.0.0 = 0.0.0.0: match, prefix /0

(2) 0.0.0.0 / 1

dest	10001110.11111010.11000011.01001110
mask	10000000.00000000.00000000.00000000
AND	**10000000.00000000.00000000.00000000**

128.0.0.0 is not 0.0.0.0: no match

(4) 128.0.0.0 / 1

dest	10001110.11111010.11000011.01001110
mask	10000000.00000000.00000000.00000000
AND	**10000000.00000000.00000000.00000000**

128.0.0.0 = 128.0.0.0: match, prefix /1

Two routes match: (1) default /0 and (4) 128.0.0.0/1.
The longest prefix wins, so route (4) is used.
Iface tun0: the packet goes into the tunnel.

After connecting to the VPN

VPN Server: 198.51.100.7
YouTube: 142.250.195.78
Client IP: 192.10.90.100

```
$ route -n
Kernel IP routing table
```

Destination	Gateway	Genmask	Flags	Metric	Iface	
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0	(1)
0.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(2)
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0	(3)
128.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(4)
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0	(5)
198.51.100.7	192.168.1.1	255.255.255.255	UGH	0	wlan0	(6)

0.0.0.0/1 = 0.0.0.0 to 127.255.255.255
128.0.0.0/1 = 128.0.0.0 to 255.255.255.255

Packet has been copied on tun0 interface.

Step 2: OpenVPN encrypts it
The OpenVPN client reads the packet from tun0, encrypts it, and wraps it in a **new** packet addressed to the VPN server, over UDP port 1194:
Outer packet: 192.10.90.100 → 198.51.100.7 (UDP 1194, payload = encrypted inner packet)

After connecting to the VPN

VPN Server: 198.51.100.7
YouTube: 142.250.195.78
Client IP: 192.10.90.100

$ route -n
Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Iface	
0.0.0.0	192.168.1.1	0.0.0.0	UG	600	wlan0	(1)
0.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(2)
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0	(3)
128.0.0.0	10.8.0.1	128.0.0.0	UG	0	tun0	(4)
192.168.1.0	0.0.0.0	255.255.255.0	U	600	wlan0	(5)
198.51.100.7	192.168.1.1	255.255.255.255	UGH	0	wlan0	(6)

Outer packet: 192.10.90.100 → 198.51.100.7

0.0.0.0/1 = 0.0.0.0 to 127.255.255.255
128.0.0.0/1 = 128.0.0.0 to 255.255.255.255

Step 3: the routing table is used again for the outer packet
The kernel looks up for **dest: 198.51.100.7**

What if we don't have rule 6?

Row 6 has the longest prefix (/32), so it wins: send via gateway 192.168.1.1 out of **wlan0**.
Without row 6, this packet would match row 4 and go back into tun0 in a loop.

Final packet: 192.10.90.100 to 198.51.100.7

At the server

VPN Server: 198.51.100.7
YouTube: 142.250.195.78
Client IP: 192.168.90.100

$ route -n
Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Iface
0.0.0.0	198.51.100.1	0.0.0.0	UG	0	eth0
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0

Packet arrived: 192.168.90.100 to 198.51.100.7

Step 1: Server directs the packet to the OpenVPN application.
Step 2: OpenVPN checks the auth tag and packet-id and decrypts.
Step 3: It checks that src IP is 10.8.0.6 belongs to that client. Drops the packet if does not match.
Step 4: It then writes the packet to **tun0.**
Step 5: The kernel sees a packet that arrived on **tun0** with src 10.8.0.6 and dest 142.250.195.78.
Step 6: The kernel checks the routing table and finds the best rule as default rule that is from **eth0**.
Uses IP-forwarding to forward packet from **tun0** to **eth0**.
Step 7: NAT stores the src IP 10.8.0.6 and changes it to the 198.51.100.7. The packet goes out.

From YouTube back to server and then to client

VPN Server: 198.51.100.7
YouTube: 142.250.195.78
Client IP: 192.168.90.100

$ route -n
Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Iface
0.0.0.0	198.51.100.1	0.0.0.0	UG	0	eth0
10.8.0.0	0.0.0.0	255.255.255.0	U	0	tun0

Return path (YouTube to client)

1. YouTube replies to 198.51.100.7. The packet arrives on eth0.

2. NAT is reversed. The tracking table matches it to the earlier connection and rewrites the destination back to 10.8.0.6.

3. The kernel routes it. 10.8.0.6 matches 10.8.0.0/24 on tun0, so the packet goes into tun0.

4. OpenVPN reads it from tun0. It looks up which client owns 10.8.0.6 in its internal routing table, which it builds from the addresses it assigned. It encrypts with that client's keys, adds the header and sends it (Client: 192.0.2.25).

5. The home router reverses its own NAT and the packet reaches the laptop, as in step 5 of the earlier slides.

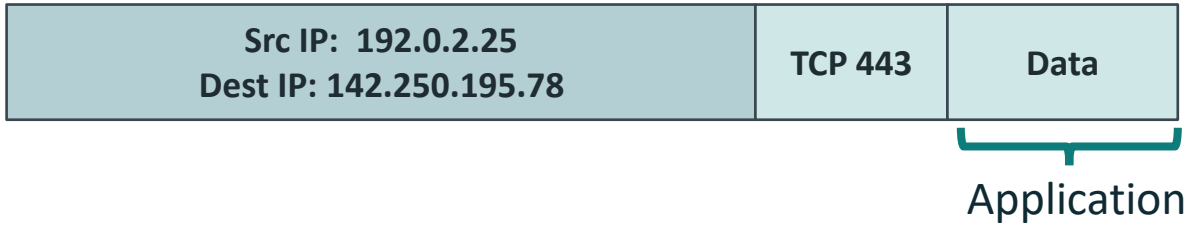
The YouTube packet, without and with a VPN

YouTube: 142.250.195.78
VPN Server: 198.51.100.7

Original packet Added by VPN Outer wrapper Address rewritten

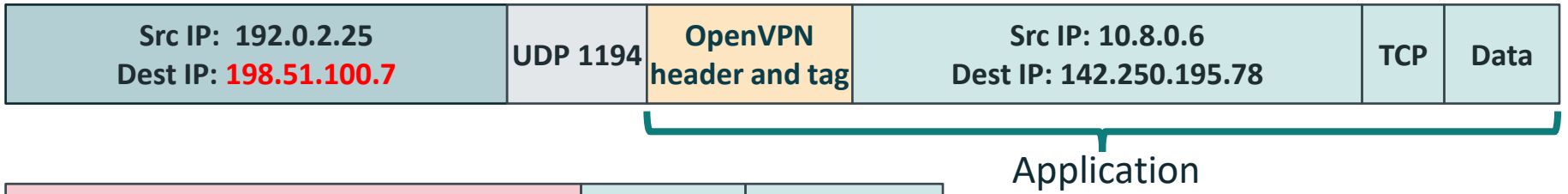
Without VPN: one IP header, destination visible

1 Leaves the PC
on wlan0



With VPN: the original packet hides inside a new outer packet

1 Leaves the PC
on wlan0, encrypted



2 Arrives at YouTube
VPN server decrypted



Without VPN: the Wi-Fi and ISP see us talking to 142.250.195.78, and YouTube sees our home IP.
With VPN: the path only sees UDP to 198.51.100.7, and YouTube sees the VPN server.

How the packets will flow when you are behind the NAT?

The YouTube packet, without and with a VPN (With NAT)

YouTube: 142.250.195.78

Original packet Added by VPN Outer wrapper Address rewritten

Without VPN: one IP header, destination visible

1 Leaves the PC
on wlan0

Src IP: 192.168.1.50 Dest IP: 142.250.195.78	TCP 443	Data

2 After home router
NAT rewrites the source

Src IP: 192.0.2.25 Dest IP: 142.250.195.78	TCP 443	Data

Source rewritten by NAT

Application

With VPN: the original packet hides inside a new outer packet

1 Leaves the PC
on wlan0, encrypted

Src IP: 192.168.1.50 Dest IP: 198.51.100.7	UDP 1194	OpenVPN header and tag	Src IP: 10.8.0.6 Dest IP: 142.250.195.78	TCP	Data

2 After home router
NAT on the outer source

Src IP: 192.0.2.25 Dest IP: 198.51.100.7	UDP 1194	OpenVPN header and tag	Src IP: 10.8.0.6 Dest IP: 142.250.195.78	TCP	Data

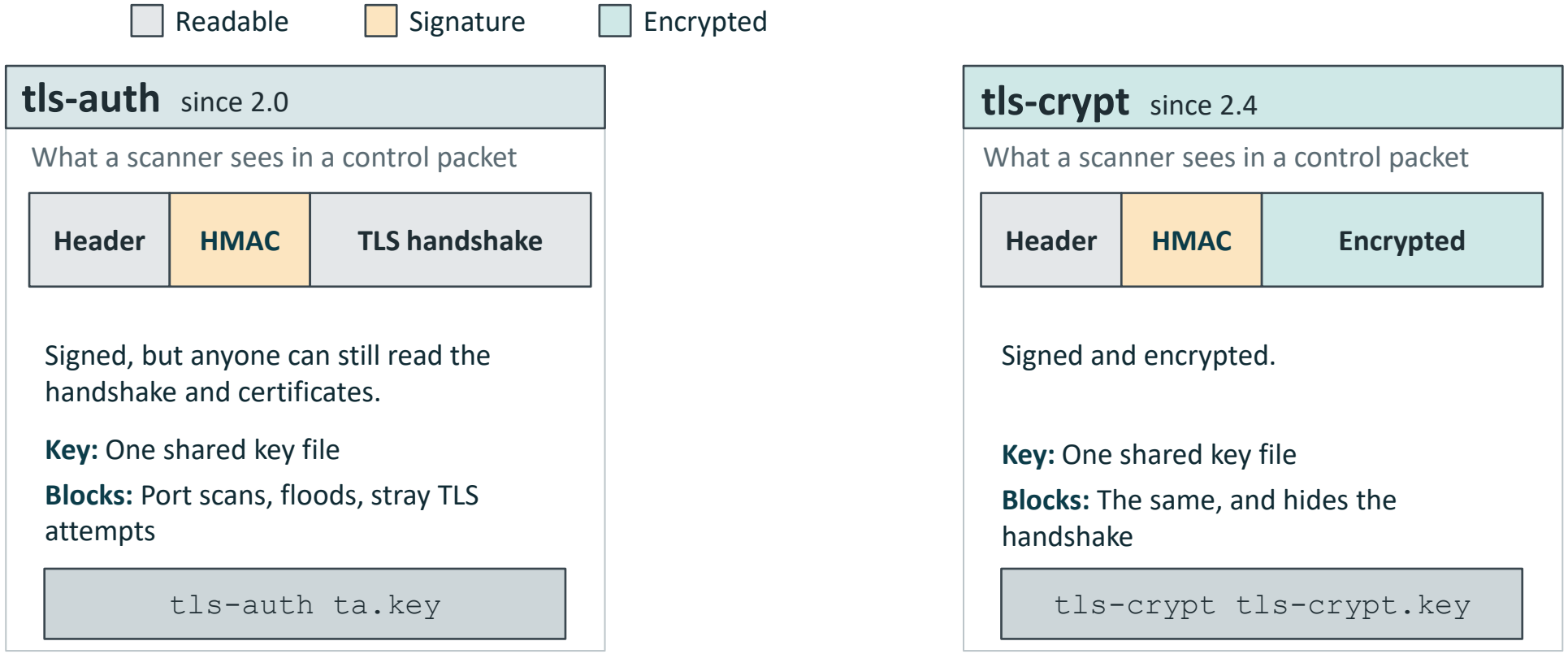
Application

3 Arrives at YouTube
VPN server decrypted and NATed

Src IP: 198.51.100.7 Dest IP: 142.250.195.78	TCP 443	Data

Can a censor/adversary fingerprint that a server is running OpenVPN?

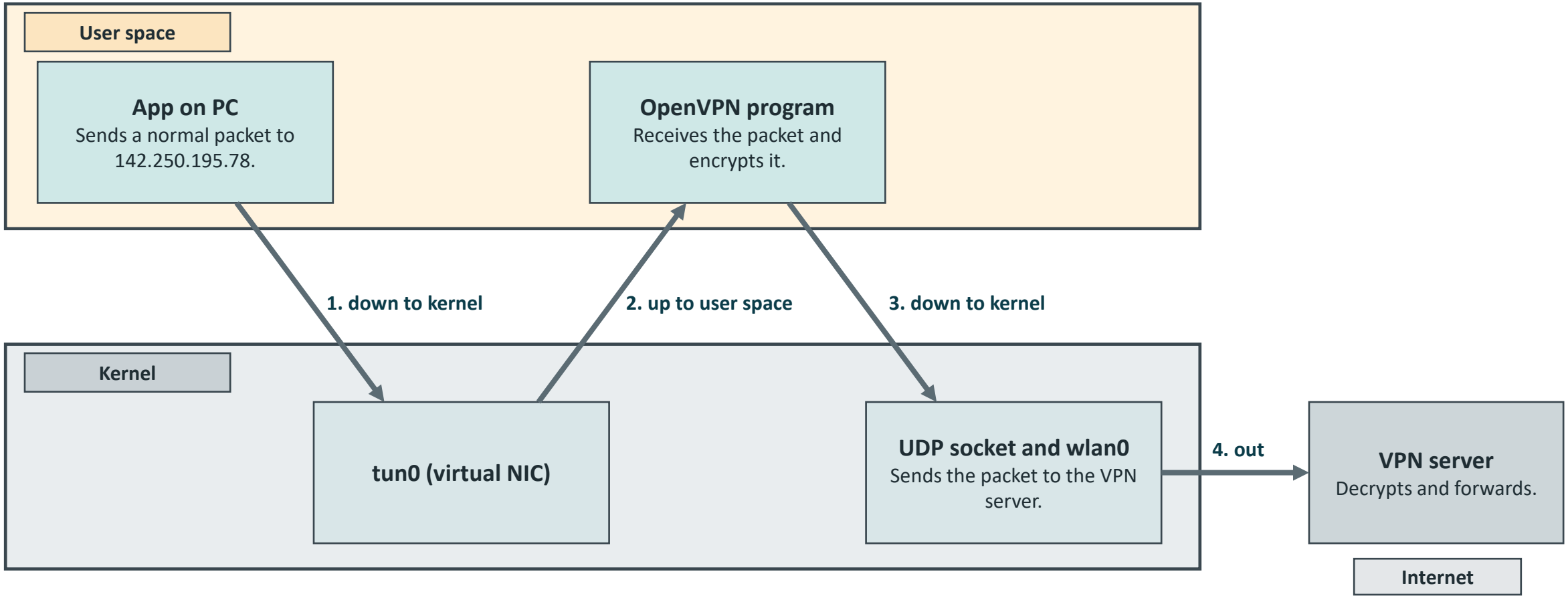
tls-auth and tls-crypt



- Only required for the control channel.
- Data channel is protected by the session derived by the TLS from control channel.

How a packet travels through OpenVPN

YouTube: 142.250.195.78



Why it can be slower than WireGuard?

- OpenVPN runs in user space.
- Every packet is copied from the kernel up to the OpenVPN program, encrypted there, and copied back down to the kernel to be sent.

OpenVPN Conf

Client.ovpn

```
client
dev tun
proto udp
remote 198.51.100.7 1194

tls-version-min 1.2

# Certificates and keys
ca   ca.crt
cert client1.crt
key  client1.key
remote-cert-tls server
tls-crypt tls-crypt.key
.
.
.
```

OpenVPN: Strengths and Limitations

Strengths

- Can bypass the firewalls (TCP 443)
- Very configurable: certificates, shared keys, MFA, plugins
- Mature, widely reviewed, runs everywhere

Limitations

- Runs in user space, so slower than WireGuard or IPsec
- Big code base, setup can get complicated
- TCP mode can be slow (TCP inside TCP)



WireGuard

WireGuard overview

~4,000

lines of code in the Linux kernel module

5.6

Linux kernel version (2020) that added it

1 RTT

one round trip for connection establishment, then data flows

WireGuard: Next Generation Kernel Network Tunnel (Jason A. Donenfeld, NDSS 2017)

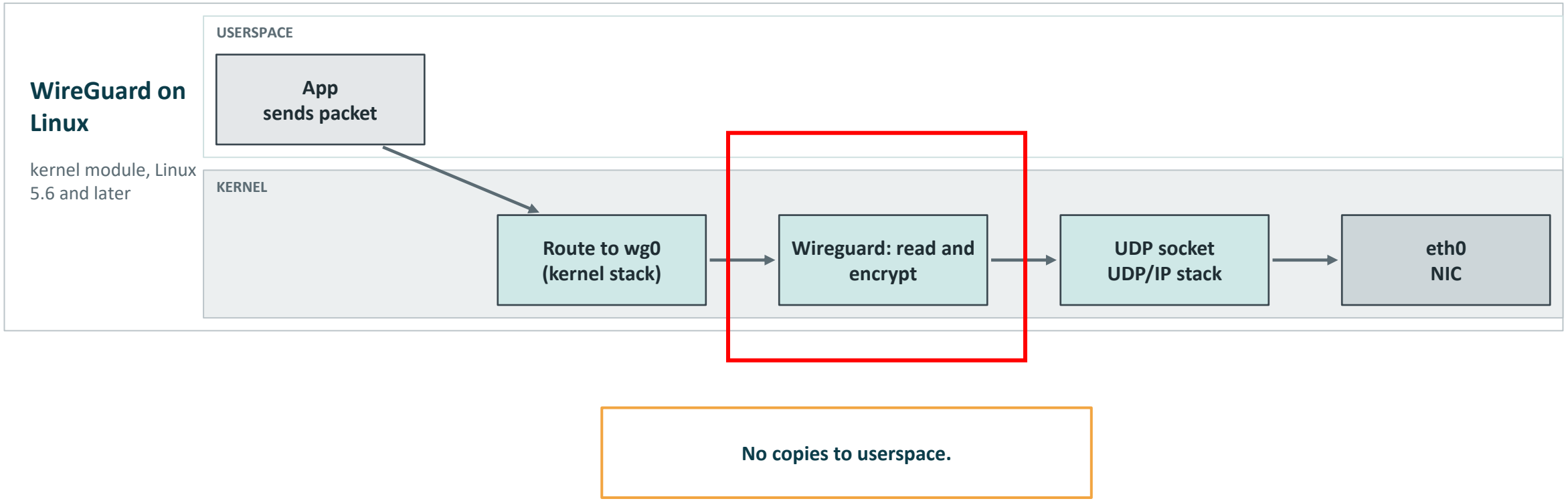
WireGuard overview

Fixed ciphers, nothing to negotiate

Noise IKpsk2	Handshake using Noise Protocol Framework	Noise requires 1 round trip to complete handshake. TLS 1.2/1.3 requires certificate and multiple packet exchanges.
ECDH using Curve25519	Elliptic-curve Diffie-Hellman	ECDH requires smaller key-size to provide better security. DH requires comparatively bigger key-size for the same level of security.
ChaCha20-Poly1305	Cipher for encryption and authentication	ChaCha20 Fast in plain software, even on phones and routers without AES hardware instructions. Uses only **add, rotate and XOR**. AES-GCM is very fast with AES-NI, but WireGuard prefers one cipher that is good everywhere.
BLAKE2s	Fast hashing	BLAKE2s is faster than SHA-256. The choice is about speed and simplicity.
KDF	Derives session keys	Same that other protocol uses.

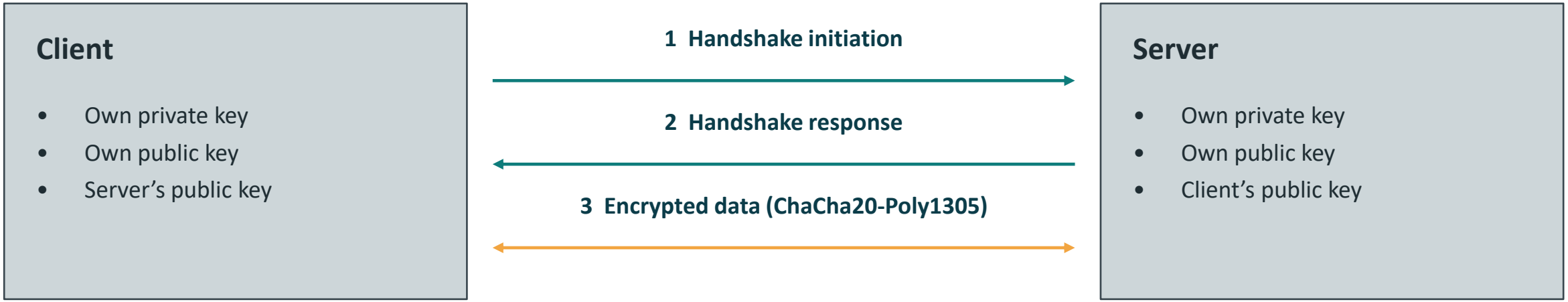
All fixed algorithms. Nothing is negotiated.

Why WireGuard runs in the kernel



It works in the kernel and the ciphers are fixed.

WireGuard:



Strengths

- Tiny code, easy to review
- Fast, runs inside the kernel
- Doesn't answer unknown senders
- Keys change about every 2 minutes

Limitations

- UDP only, so it can be blocked
- No built-in user login or MFA

WireGuard config

Client: C_priv, C_pub
Server: S_priv, S_pub

VPN Server: 203.0.113.7
Port: 51820

Client: wg0.conf

```
[Interface]
PrivateKey =<client static private key>
Address = 10.0.0.2/32

[Peer]
# this peer is the server
PublicKey = <server static public key>
Endpoint = 203.0.113.7:51820
AllowedIPs = 0.0.0.0/0
# 0.0.0.0/0: send everything
PersistentKeepalive = 25
```

Server: wg0.conf

```
[Interface]
PrivateKey = <server static private key>
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
# this peer is the client
PublicKey = <client static public key>
AllowedIPs = 10.0.0.2/32
# no Endpoint: learned from the
# first valid packet peer client sends
```

Field	Meaning
PrivateKey	The Curve25519 private key. It never leaves the machine. The matching public key is the machine's identity.
Address	The IP given to the wg0 interface. The server uses the whole subnet (10.0.0.1/24), and a client usually its own address (/32).
PublicKey	Identifies the peer. There are no usernames or certificates. If the key matches, it is that peer.
Endpoint	The IP and port to send to. It is optional, and a machine (server) that only waits for connections doesn't need one for the other side.
AllowedIPs	The addresses that belong to this peer. This is the most important field, explained below.
PersistentKeepalive	Sends an empty packet every N seconds. Without it, nothing is sent while idle.
ListenPort	The UDP port to accept handshakes on. A server sets it (51820 by default). A client can omit it and the kernel picks a random port.

Handshake initiation

Sent in plaintext
Encrypted (ChaCha20-Poly1305)
MAC (BLAKE2s)

Client: wg0.conf

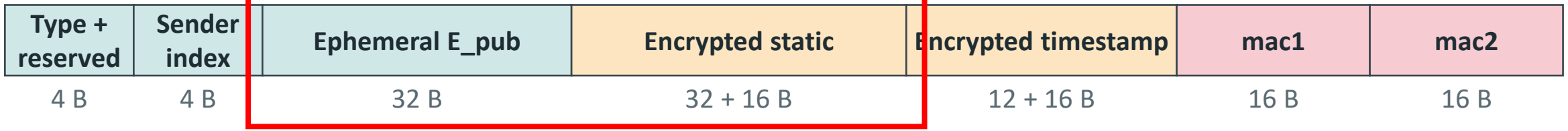
```
[Interface]
PrivateKey =<client static private key>
Address = 10.0.0.2/32

[Peer]
# this peer is the server
PublicKey = <server static public key>
Endpoint = 203.0.113.7:51820
AllowedIPs = 0.0.0.0/0
# 0.0.0.0/0: send everything
PersistentKeepalive = 25
```

Server: wg0.conf

```
[Interface]
PrivateKey = <server static private key>
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
# this peer is the client
PublicKey = <client static public key>
AllowedIPs = 10.0.0.2/32
# no Endpoint: learned from the
# first valid packet it sends
```



At the client

The client sends its public key encrypted with the key **K**.

Where K comes from:

1. Client knows the Server's static public key (PublicKey), as S_pub
2. Client selects Ephemeral key (E_priv, E_pub).
3. The client computes DH(E_priv, S_pub), and gets the shared secret key.
4. This shared secret key is used in the KDF to get the key **K.**

Deffie-Hellman Key Exchange

C_Priv, C_Pub
E_C_Priv, E_C_Pub

From conf: S_Pub

Client

g, p

Server

S_Priv, S_Pub
E_S_Priv, E_S_Pub

From conf: C_Pub

Private = x

Public X = g ^ x mod p

Private = y

Y = g ^ y mod p

X



Secret Key = Y ^ x mod p

Secret Key = X ^ y mod p

x = E_Priv

X = E_Pub

y = S_Priv

Y = S_Pub

Handshake response

Client: wg0.conf

```
[Interface]
PrivateKey = <client private key>
Address = 10.0.0.2/32

[Peer]
# this peer is the server
PublicKey = <server static public key>
Endpoint = 203.0.113.7:51820
AllowedIPs = 0.0.0.0/0
# 0.0.0.0/0: send everything
PersistentKeepalive = 25
```

Server: wg0.conf

```
[Interface]
PrivateKey = <server private key>
Address = 10.0.0.1/24
ListenPort = 51820

[Peer]
# this peer is the client
PublicKey = <client static public key>
AllowedIPs = 10.0.0.2/32
# no Endpoint: learned from the
# first valid packet it sends
```

- Sent in plaintext
- Encrypted (ChaCha20-Poly1305)
- MAC (BLAKE2s)

At the server

Server side: recovering the client's public key

1. The server reads the client's ephemeral public key (E_pub) from the packet. It is sent in plaintext.
2. The server has its own static private key (PrivateKey in its wg0.conf).
3. The server computes DH(PrivateKey, E_pub). This gives **the same shared secret** as the client's DH(E_priv, server PublicKey).
4. The server runs the same KDF on it and gets the same key **K**.
5. It decrypts the "encrypted static" field with **K** (ChaCha20-Poly1305). The tag check passes only if **K** is correct.
6. The result is the client's public key. The server looks it up among the PublicKey entries of its [Peer] blocks. If there is no match, it drops the packet.

Response Packet:

Type + reserved	Sender index	Receiver index	Ephemeral Er_pub	Encrypted nothing	mac1	mac2
4 B	4 B	4 B	32 B	0 + 16 B tag	16 B	16 B

DH is prone to MITM.
Can MITM happen in WireGuard too?

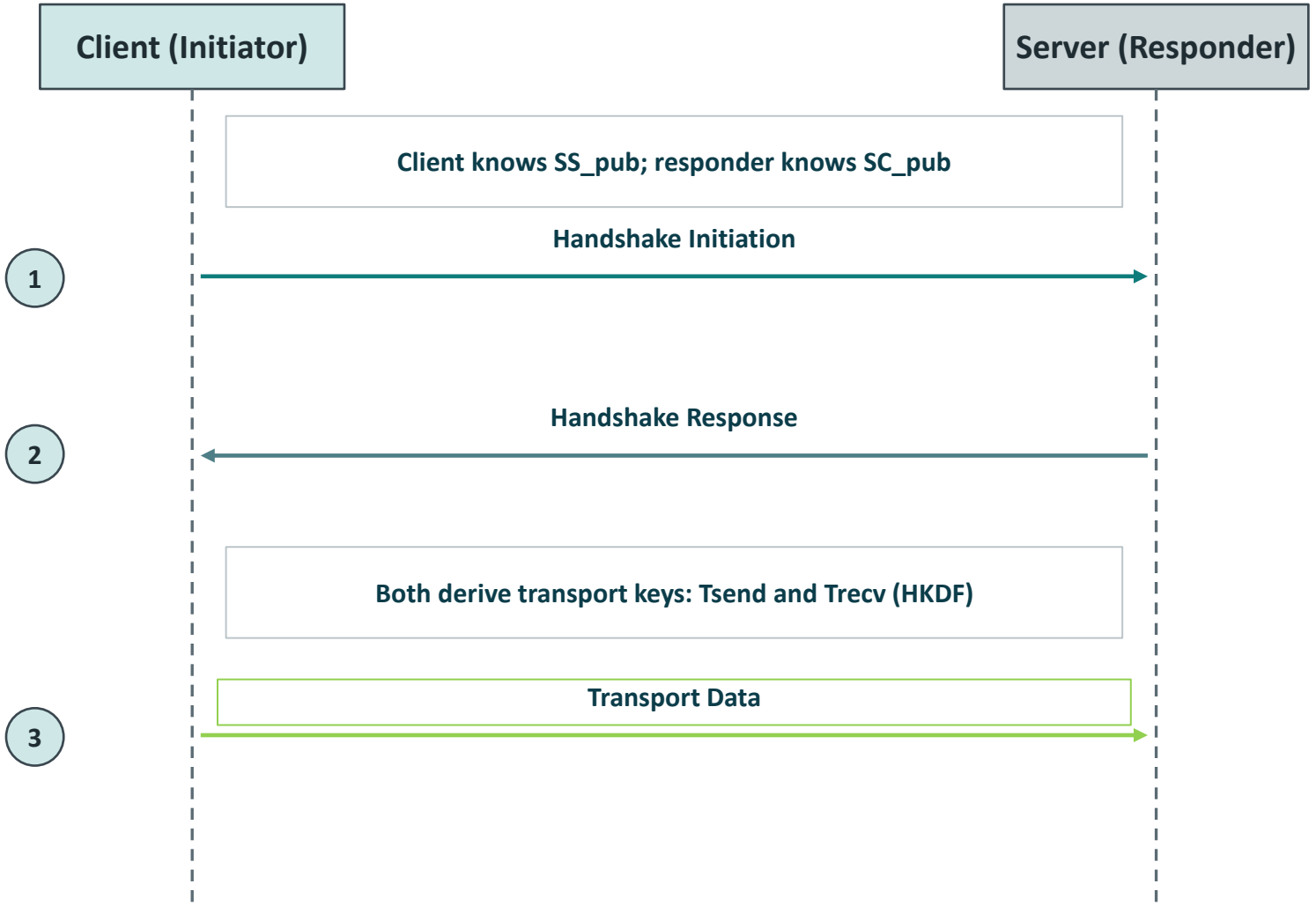
No, since the public component of the server is known to the client beforehand.

The four Diffie-Hellman computations

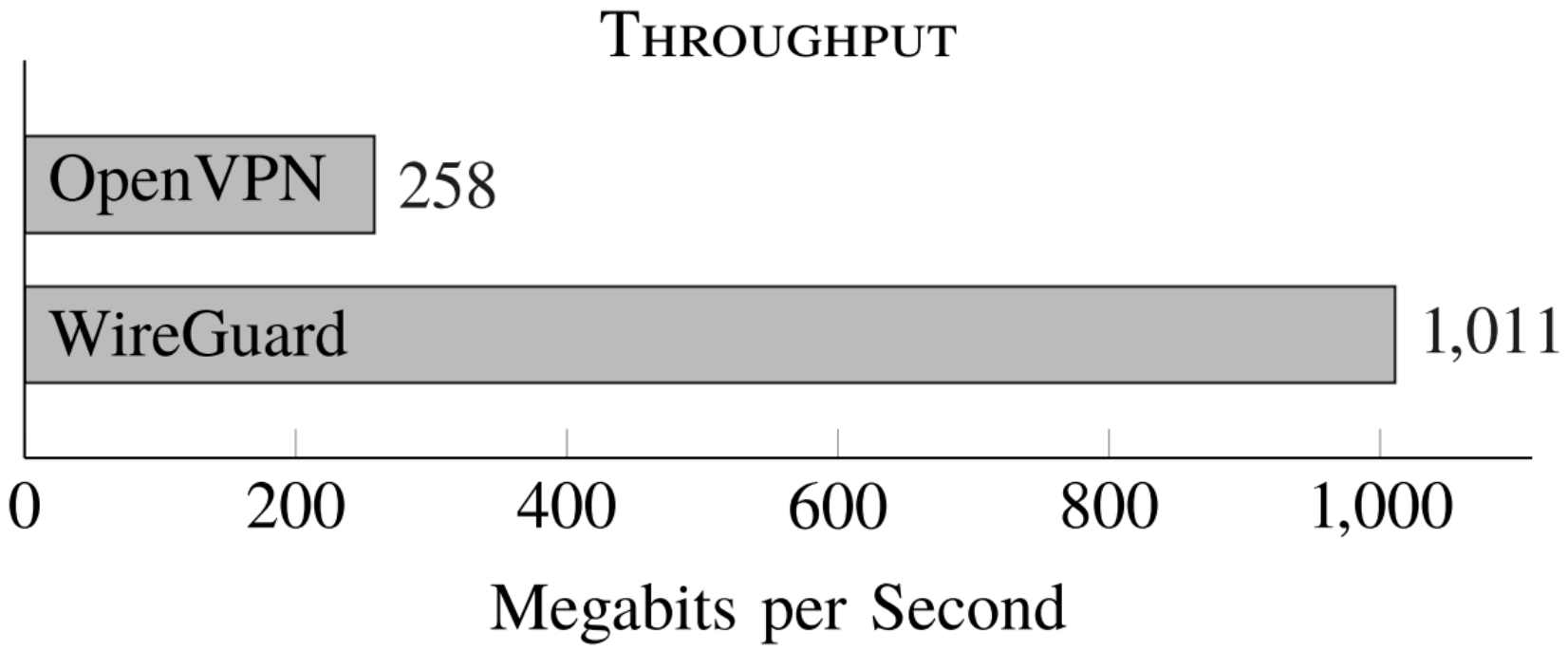
DH	Client computes	Server computes	From	Why it is there
1	`EC_priv, SS_pub`	`SS_priv, EC_pub`	Initiation	Encrypts the client's public key. Only the real server can read it.
2	`SC_priv, SS_pub`	`SS_priv, SC_pub`	Initiation	Authenticates the client: it needs the client's private key. Also keys the encrypted timestamp to remove replays.
3	`EC_priv, ES_pub`	`ES_priv, EC_pub`	Response	Forward secrecy: it involves only ephemeral keys, which are deleted afterwards.
4	`SC_priv, ES_pub`	`ES_priv, SC_pub`	Response	Ties the session keys to the client's identity as well.

E = Ephemeral, S = Static
C = Client, S = Server

WireGuard handshake, message by message



Throughput Comparison



WireGuard: Next Generation Kernel Network Tunnel (Jason A. Donenfeld, NDSS 2017)

You have learned VPNs and its working successfully.